

Python Programming Fundamentals

A practical 4-page reference document for Python development and AI applications.

1. Introduction to Python

Python is a high-level, interpreted, general-purpose programming language known for readable syntax and a large ecosystem. It is widely used for web development, automation, data analysis, machine learning, artificial intelligence, scripting, and testing.

Key Characteristics

- Simple and readable syntax
- Dynamically typed
- Supports object-oriented, procedural, and functional programming
- Large standard library and third-party ecosystem
- Cross-platform and open source

2. Variables and Data Types

A variable is a name that refers to a value. Python determines the type of a value at runtime, so explicit type declarations are generally not required.

Common types: int, float, str, bool, list, tuple, set, and dict.

Example: age = 25; name = 'Alex'; scores = [85, 91, 78].

3. Control Flow

Control-flow statements determine which instructions execute and how many times they execute. Python provides conditional statements and loops.

Conditional Statements

An if/elif/else structure evaluates conditions and chooses the appropriate block of code.

Loops

A for loop iterates over a sequence, while a while loop continues while a condition remains true.

4. Functions

A function is a reusable block of code that performs a specific task. Functions improve modularity, readability, testing, and maintainability.

Important concepts

- Parameters and arguments
- Return values
- Default and keyword arguments
- Variable scope

5. Object-Oriented Programming

Object-oriented programming organizes software around objects that combine data and behavior. Python supports classes, objects, inheritance, encapsulation, and polymorphism.

Core OOP concepts

- Class — blueprint for creating objects
- Object — instance of a class
- Encapsulation — organizing data and behavior
- Inheritance — deriving a class from another class
- Polymorphism — common interfaces with different implementations

6. Modules, Packages, and Virtual Environments

A module is a Python file containing reusable code. A package is a collection of modules organized into a directory structure. Virtual environments isolate dependencies between projects.

Why use a virtual environment?

Different projects may require different library versions. A virtual environment reduces dependency and version conflicts.

7. File Handling

Python can read and write text files and work with JSON and CSV. Context managers are commonly used so files are closed automatically.

Typical workflow

- Open the file
- Read or write content
- Process the data
- Close the resource automatically

8. Exceptions and Error Handling

Exceptions are runtime events that interrupt normal execution. Python provides try, except, else, and finally for handling errors.

Good error handling should provide useful information, avoid hiding unexpected problems, and allow recovery when appropriate.

9. Python in AI and Machine Learning

Python is widely used in AI because of libraries and frameworks such as NumPy, pandas, scikit-learn, PyTorch, TensorFlow, and LLM application frameworks. A typical AI workflow may load data, preprocess it, create features or embeddings, run a model, and evaluate the result.

10. Practical Python Development Workflow

A maintainable Python project separates responsibilities into modules and uses dependency management and version control.

Recommended workflow

1. Create a project directory.
2. Create and activate a virtual environment.
3. Install required dependencies.
4. Organize reusable code into modules.
5. Add tests for important functionality.
6. Track source code with Git.
7. Document how to install and run the project.

11. Python Concepts to Remember

For interviews and practical development, focus on lists versus tuples, mutable versus immutable objects, scope, copying, exception handling, OOP principles, iterators, generators, decorators, and package management.

12. Example AI Use Case

Consider a customer-review analysis application. Python can load review data, clean text, generate embeddings, store vectors in a vector database, retrieve semantically similar reviews, and send relevant context to an LLM. FastAPI can expose APIs, while Streamlit can provide a demonstration interface.

Summary

Python provides the programming foundation for many modern AI applications. Strong fundamentals make it easier to work with AI frameworks, APIs, databases, vector stores, and LLM systems.